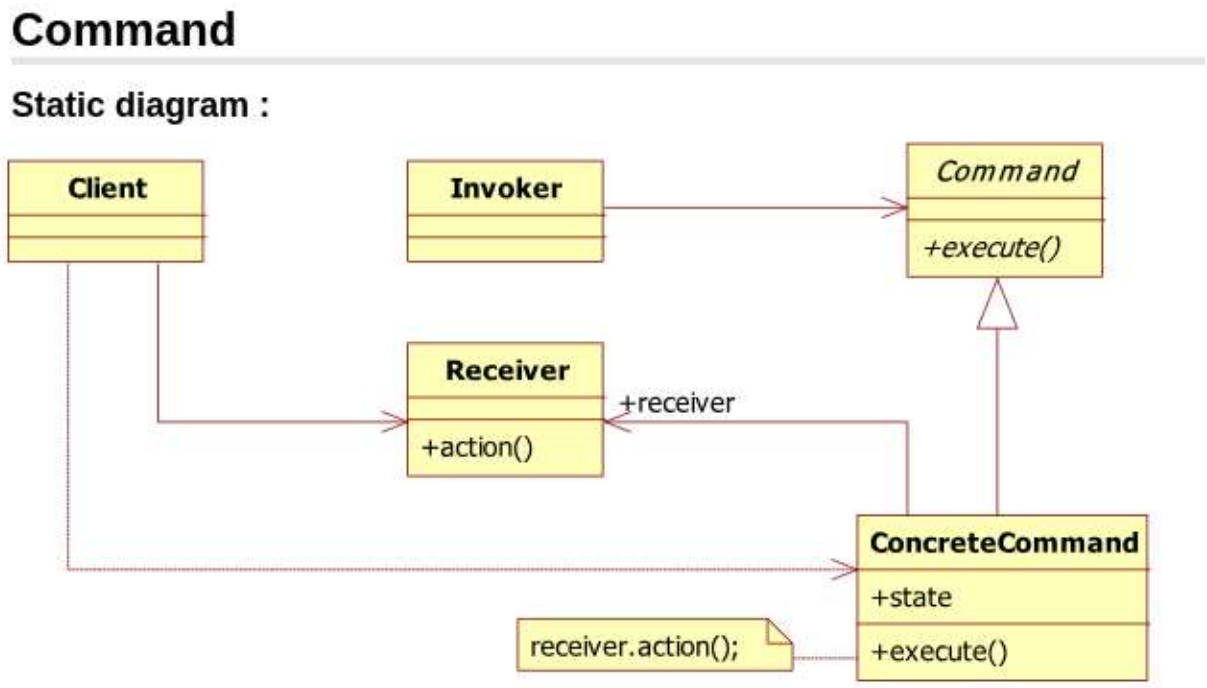
Résumé design pattern Command

Le pattern commande est un patron de conception comportemental qui prend une action à effectuer et la transforme en un objet autonome qui contient tous les détails de cette action. Cette transformation permet de paramétrer des méthodes avec différentes actions, planifier leur exécution, les mettre dans une file d'attente, d'annuler des opérations effectuées et d'enlever le fort couplage de classes.

Cela permet aussi dans certains cas, comme les commandes sont stockées, de réexécuter les commandes après un plantage du système pour revenir au même état qu'avant.

On utilise donc le pattern command quand :

- on veut avoir une action (une commande) qui peut être activée de plusieurs façons (par un bouton, un raccourci clavier etc.)
- on veut un historique undo/redo
- on veut enregistrer les tâches



L'invoker envoie les demandes (dans notre exemple du restaurant c'est le serveur), mais ce n'est généralement pas lui qui les a créées

L'interface Command déclare habituellement une seule méthode pour exécuter la commande, ou également une méthode undo.

Les ConcreteCommands implémentent plusieurs types de demandes. Une commande concrète n'accomplis pas la tâche elle-même. Elle transmet l'appel aux objets de la logique métier.

La classe Receiver porte la logique métier. À peu près n'importe quel objet peut prendre le rôle de receiver. La majorité des commandes se contentent de passer la demande au récepteur et ce dernier se charge du traitement.

Le Client crée et configure les objets de la commande concrète. Il doit passer tous les paramètres de la demande (dont l'instance du récepteur) dans le constructeur de la commande. Ensuite, la commande qui en résulte peut-être associée avec un ou plusieurs demandeurs.

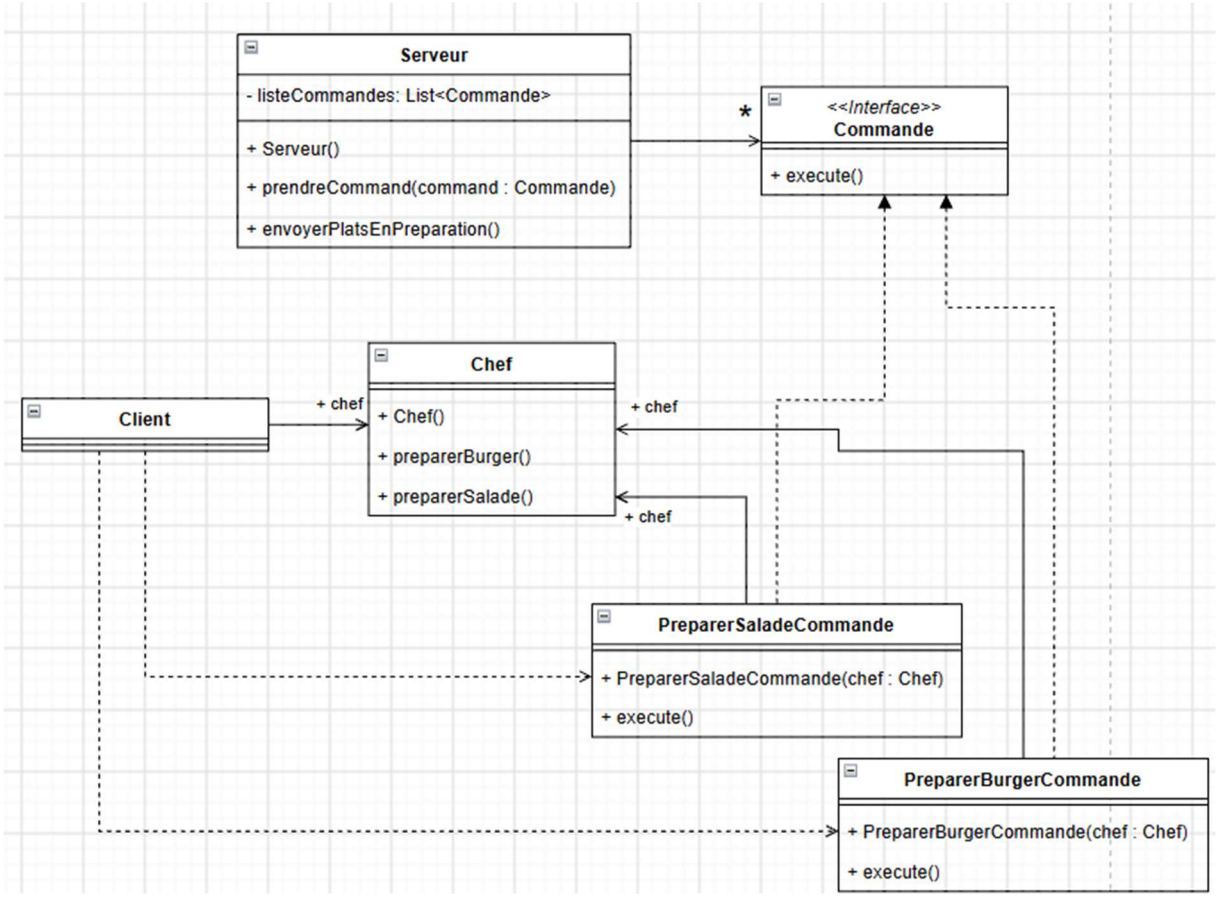
Prenons l'exemple d'un serveur dans un restaurant qui prend des commandes et doit ensuite les donner au chef en cuisine pour qu'il les prépare. Le serveur doit alors prendre des commandes différentes (ici notre restaurant fait uniquement des burgers et des salades), puis les transmettre au chef pour qu'il les prépare.

On peut traduire en terme métier et réaliser ce tableau :

Nom du design pattern	**Nom dans notre exemple**
Client	Client
Receiver	Chef
Command	Command
Invoker	Serveur
Concrete Command 1	PreparerBurgerCommande
Concrete Command 2	PreparerSaladeCommande

Avantages	Inconvénients
Découplage fort de l'invoker et receiver	Chaque action spécifique nécessite une nouvelle classe concrète, alourdissant la structure
Ajout de nouvelles actions sans toucher au code existant	Ajout de la couche d'abstraction intermédiaire qui peut augmenter la complexité
Gestion undo/redo	

Avec notre pattern cela se modélise :



Le pattern Command et les principes SOLID :

- SRP - On peut découpler les classes qui appellent des traitements, de celles qui les exécutent, donc chaque classe a une responsabilité.
- OCP – On peut ajouter de nouvelles commandes dans le programme sans changer le code client existant. Le code est fermé aux modifications mais ouvert à l'extension.
- LSP - Toutes les commandes concrètes implémentent la même interface commune `Commande`. N'importe quelle commande concrète peut être substituée à une autre et injectée dans l'Invocateur sans altérer ni casser le comportement du programme.
- ISP – L'interface commande est minimaliste, donc les classes clientes ne dépendent pas de méthodes non utilisées.

- DIP – Invoker ne dépend pas directement des classes concrètes Receiver, mais d'une abstraction (Commande). Les modules de haut et bas niveau dépendent tous deux de cette interface.